UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

Facultad de Ciencias

Sistemas Operativos

Aislamiento con MPU y Estrategias de Recuperación en Sistemas Críticos

---

Práctica 7

Presenta:
Rojo Peña Manuel Ianluck

Profesor:
Javier León Cotonieto

Ayudante laboratorio:
Adrián Martínez Manzo

Grupo: 7004, 2026-2

Fecha de entrega:
27 abril, 2026

Introducción

En esta práctica exploramos uno de los problemas más delicados del diseño de sistemas embebidos de misión crítica qué ocurre cuando el software falla en tiempo de ejecución y cómo el sistema puede sobrevivir a ese fallo de forma controlada. Para ello estudiamos el papel de la **Unidad de Protección de Memoria** (MPU), que permite delimitar regiones de memoria con permisos estrictos de lectura, escritura y ejecución.

La premisa fundamental que recorre toda la práctica es la siguiente, la MPU no elimina los errores, sino que los contiene, cuando una tarea intenta acceder a una región de memoria que no le pertenece, la MPU detiene la ejecución y transfiere el control a un manejador de excepciones conocido como **HardFault Handler**. Lo que ese handler haga a continuación determina si el sistema muere, sobrevive degradado o renace limpio, esa decisión arquitectónica es el núcleo de lo veremos en esta práctica.

El reporte se organiza en dos fases. En la **Fase A** donde analizamos cinco escenarios reales de fallo en sistemas de misión crítica, desde un gateway industrial hasta un satélite en órbita profunda, y para cada uno seleccionamos y justificamos la estrategia de recuperación más adecuada entre las cinco opciones dadas: *Patrón de Caja Negra*, *Reinicio Duro* al *Estado Seguro*, *Degradación Elegante*, *Micro-Reinicio* de Tarea y *Checkpointing* con *Rollback*.

En la **Fase B** pasamos del análisis teórico a la implementación práctica, utilizando la placa Raspberry Pi Pico, configuraremos una trampa de memoria mediante la MPU y probamos cuatro estrategias de recuperación distintas sobre hardware real, *Rollback* por software, *Degradación Elegante*, *Micro-Reinicio* de Tarea y *Reinicio Duro* por *Watchdog*.

Fase A: Análisis de Escenarios y Toma de Decisiones

Antes de analizar los escenarios, es necesario establecer el marco conceptual que define el comportamiento de la MPU y las estrategias de recuperación disponibles. La **Unidad de Protección de Memoria** (MPU, por sus siglas en inglés) es un componente de hardware que establece regiones de memoria con permisos específicos de lectura, escritura y ejecución. Cuando una tarea intenta acceder a una dirección fuera de sus permisos asignados, la MPU interrumpe inmediatamente la ejecución y transfiere el control a un manejador de excepciones conocido como **HardFault Handler**. Es importante subrayar que la MPU no previene los errores lógicos en sí mismos, sino que impide que dichos errores se propaguen y corrompan el estado de otras tareas o del sistema operativo en su conjunto.

La decisión arquitectónica crítica reside en qué instrucciones se programan dentro de ese manejador de excepciones. La elección determina si el sistema sobrevivirá al fallo o se detendrá de forma definitiva. A continuación se describen las cinco estrategias de recuperación contempladas en esta práctica.

Estrategias de Recuperación

A) Patrón de Caja Negra (Registro de Telemetría)

Esta estrategia asume que el error es fatal e irrecuperable durante el tiempo de ejecución, pero reconoce que la información del fallo es invaluable para el proceso de desarrollo y diagnóstico posterior. Cuando la MPU detecta una violación, el *HardFault Handler* detiene la ejecución normal y ejecuta una rutina mínima y ultra-segura que extrae el estado completo del procesador, el *Program Counter* (PC), el *Link Register* (LR), los registros de estado de la MPU y el volcado del stack.

El objetivo no es salvar el sistema, sino escribir estos datos en una memoria no volátil (como una EEPROM o una partición flash dedicada) antes de que el sistema muera o se congele. Al recuperar esa memoria posteriormente, los desarrolladores pueden identificar con precisión qué instrucción en ensamblador provocó la violación, permitiendo una depuración forense que de otro modo sería imposible.

B) Reinicio Duro al Estado Seguro (Fail-Safe / Hardware Reset)

Es la táctica de mayor contundencia y se utiliza cuando la integridad del sistema está tan comprometida que continuar ejecutando código representa un peligro físico real, como en robótica, aviónica o control de motores de alto voltaje. Una vez que la MPU atrapa el error crítico, el software fuerza un reinicio a nivel de hardware, lo cual se logra escribiendo en un registro de control del microcontrolador (por ejemplo, en arquitecturas `ARM Cortex-M`, escribiendo en el registro `AIRCR`) o dejando que el *Watchdog Timer* expire de forma intencional.

El propósito es devolver todo el hardware a un estado eléctrico conocido y seguro, donde los pines `GPIO` regresan a alta impedancia, los actuadores se apagan y el sistema reinicia la secuencia de arranque desde cero, eliminando cualquier corrupción persistente en la RAM.

C) Degradación Elegante (Graceful Degradation)

Esta estrategia es donde el diseño arquitectónico modular demuestra su valor. Parte del supuesto de que el sistema está compuesto por múltiples subsistemas y que no todos tienen igual criticidad para la misión principal. Si la MPU detecta que una tarea no crítica (como el hilo encargado de actualizar la pantalla LCD o de registrar datos en una tarjeta SD) intentó corromper la memoria, el sistema operativo puede aislar ese fallo de forma quirúrgica.

La tarea problemática es desactivada y sus recursos son liberados, pero el núcleo del sistema continúa operando. Se mantiene la funcionalidad principal activa, aunque con capacidades reducidas, si un sensor primario falla, el sistema puede transicionar a un sensor secundario o a un algoritmo de menor precisión, evitando un colapso total a costa de ofrecer un servicio degradado pero seguro.

D) Micro-Reinicio de Tarea (Task Respawning)

Esta estrategia depende de que el aislamiento de memoria configurado por la MPU sea perfecto y verificable. En lugar de reiniciar todo el hardware, el sistema operativo o RTOS intercepta la excepción y, al tener certeza de que la MPU impidió que la tarea culpable corrompiera la memoria de otras tareas, procede a destruir únicamente ese hilo, libera sus recursos, limpia su stack y lo reinstancia desde cero.

Es el equivalente en sistemas embebidos a ejecutar `kill -9` sobre un proceso específico y relanzar el demonio correspondiente, sin necesidad de reiniciar el kernel completo. Resulta ideal para procesos que gestionan sockets de red o comunicaciones propensas a fallos transitorios que no afectan el control físico crítico del sistema.

E) Checkpointing y Rollback

Es una estrategia transaccional aplicada a la memoria de ejecución, ampliamente utilizada en bases de datos y simulaciones complejas, adaptada aquí para sistemas de alta disponibilidad. El sistema guarda periódicamente una "fotografía", de su estado seguro (registros críticos, variables de estado, configuración de la máquina de estados) en una región de memoria protegida. Si ocurre una violación de MPU, el sistema aborta la operación corrupta y restaura la memoria y los registros desde el último checkpoint válido.
El objetivo es deshacer el error en lugar de eliminarlo por completo. En vez de reiniciar desde cero o matar la tarea, el sistema retrocede en el tiempo hasta un instante donde no existía corrupción e intenta continuar la ejecución desde ese punto. Esta estrategia requiere un diseño de software muy cuidadoso para garantizar que el estado restaurado sea coherente con el mundo físico externo al sistema.

Con este marco establecido, se procede al análisis de los cinco escenarios planteados.

Caso 1: El Enrutador IoT Industrial

Un Gateway en una fábrica ejecuta dos tareas en paralelo, **Enrutamiento MQTT**, que es la tarea vital encargada de comunicar los sensores de producción, y **Servidor Web Config**, una interfaz gráfica secundaria para administración. Un ataque de red provoca un desbordamiento de búfer en la tarea del Servidor Web Config, y la MPU aísla y atrapa la amenaza.

Estrategia Seleccionada: Degradación Elegante (Graceful Degradation)

Justificación Arquitectónica: El ataque de red expuso una vulnerabilidad en el Servidor Web, que es un componente secundario y prescindible dentro de la misión del gateway. Al aplicar Degradación Elegante, el sistema operativo aísla y termina únicamente la tarea del servidor web, cierra el puerto expuesto y libera sus recursos, mientras el enrutamiento MQTT permanece intacto. La fábrica continúa operando sin interrupción.

Aplicar un Reinicio Duro (B) desconectaría todos los sensores de producción, causando un paro operativo inaceptable por el fallo de un sistema secundario. El Micro-Reinicio de Tarea (D) o el Rollback (E) serían contraproducentes en este contexto, pues si se reinicia la tarea web sin corregir la vulnerabilidad, el atacante puede relanzar el mismo *buffer overflow* de forma inmediata, generando un ciclo infinito de caídas y reinicios que saturaría los recursos del procesador.

Caso 2: El Brazo Robótico Quirúrgico

Durante una operación en curso, la MPU detecta que la tarea del controlador cinemático principal, responsable de calcular la posición de los motores en tiempo real, intentó ejecutar una instrucción corrupta. El procesador entra en estado de HardFault.

Estrategia Seleccionada: Reinicio Duro al Estado Seguro (Fail-Safe / Hardware Reset)

Justificación Arquitectónica: En sistemas de misión crítica donde hay vidas humanas en juego y se controlan actuadores físicos de alta potencia, cualquier estado inconsistente en la memoria es inaceptable.
El Reinicio Duro corta la energía de todos los motores de forma inmediata a través de los circuitos Fail-Safe del hardware, impidiendo que el brazo realice un movimiento involuntario que pueda causar daño irreversible al paciente. La Degradación Elegante (C) es inviable en este caso, ya que no es posible realizar una intervención quirúrgica sin el cálculo cinemático preciso; no existe una "funcionalidad reducida" aceptable. El Checkpointing y Rollback (E) es particularmente peligroso aquí, pues restaurar el estado de hace unos milisegundos asume que el mundo físico externo no cambió durante ese intervalo, cuando en realidad el bisturí, el tejido u otro instrumento pueden haber variado de posición. El software quedaría desincronizado de la realidad física, creando una condición de riesgo aún mayor.

Caso 3: El Dron de Exploración Agrícola

El sensor ultrasónico de un dron sufre un cortocircuito eléctrico que inunda al sistema con interrupciones descontroladas, provocando un desbordamiento de stack (*Stack Overflow*) en la tarea gestora de ese sensor. La MPU atrapa el error. Se sabe de antemano que reiniciar la tarea no servirá, pues el hardware dañado colapsará la tarea nuevamente de forma inmediata.

Estrategia Seleccionada: Degradación Elegante (Graceful Degradation)

Justificación Arquitectónica: La raíz del problema es un fallo de hardware físico, no un error de software corregible. La Degradación Elegante permite al sistema operativo instruir al planificador para que deshabilite las interrupciones del pin afectado y elimine permanentemente la tarea del sensor ultrasónico. El dron pierde la capacidad de detección de obstáculos por ultrasonido, pero el controlador de vuelo principal permanece activo, permitiendo ejecutar un aterrizaje controlado utilizando el barómetro y el módulo GPS.

Un Reinicio Duro (B) cortaría la energía de los motores mientras el dron está en el aire, lo que resultaría en un impacto inevitable. El Micro-Reinicio de Tarea (D) es inútil en este escenario porque, en cuanto la tarea renaciera y habilitara de nuevo el periférico dañado, la tormenta de interrupciones volvería a provocar el desbordamiento de stack de manera instantánea. El problema no es el software de la tarea, sino el hardware que la alimenta.

Caso 4: Satélite de Órbita Profunda

La radiación cósmica provoca un *Single Event Upset* (SEU) en la RAM del satélite, volteando un bit y corrompiendo la memoria de la tarea de Navegación Astronómica. La MPU detecta el acceso inválido y detiene la tarea. Reiniciar el cálculo desde cero implicaría tres semanas de tiempo de cómputo perdido.
Estrategia Seleccionada: Checkpointing y Rollback

Justificación Arquitectónica: En el entorno espacial, los SEUs son fallos transitorios bien documentados, el hardware físico permanece en perfecto estado y únicamente se corrompió un dato en la RAM. Dado que el cálculo astronómico representa tres semanas de trabajo computacional acumulado, resulta inaceptable descartarlo por completo. Con Checkpointing, el sistema puede restaurar la memoria desde el último punto de guardado (que podría ser de hace una hora) almacenado en una memoria resistente a radiación como MRAM o FRAM, y reanudar el cálculo desde ese instante con un costo mínimo.

Un Reinicio Duro (B) o un Micro-Reinicio de Tarea (D) purgarían la RAM por completo, destruyendo semanas de resultados computacionales sin posibilidad de recuperación. La Degradación Elegante (C) dejaría al satélite sin su sistema de navegación principal, imposibilitando cualquier maniobra de corrección orbital y convirtiendo la misión en un fracaso total.

Caso 5: La Autopsia del Desastre

Un patín eléctrico comercial sufre un HardFault durante su operación. El sistema aplica correctamente un Reinicio Duro, salvando al usuario de un posible accidente. Sin embargo, al analizar el dispositivo en el laboratorio, los desarrolladores no tienen información alguna sobre qué tarea falló ni en qué línea de código ocurrió el evento.

Estrategia que faltó ejecutar *antes* del reinicio: Patrón de Caja Negra (Registro de Telemetría)

Justificación Arquitectónica: El sistema actuó correctamente al aplicar el Reinicio Duro para proteger al usuario, pero los desarrolladores quedaron ciegos porque el reinicio borró toda la memoria volátil (SRAM). Lo que faltó fue ejecutar una rutina de Caja Negra inmediatamente antes de invocar el reinicio por hardware. El HardFault Handler debió capturar el Program Counter (PC), el Link Register (LR) y los registros de estado de la MPU, y escribirlos de forma atómica en la memoria Flash o en una EEPROM externa. Con esos datos disponibles en el laboratorio, es posible identificar con precisión qué instrucción en C o ensamblador desencadenó el fallo.

El Checkpoint/Rollback o la Degradación Elegante son técnicas diseñadas para mantener el sistema operando, no para dejar un rastro forense. Si la gravedad del fallo justificó un Reinicio Duro para salvaguardar la integridad del usuario, la única obligación secundaria del sistema en ese instante era dejar registrado su "testamento" antes de apagarse.

Fase B: Laboratorio de Resiliencia (Nexus Medical)

El firmware base simula un dispositivo médico de monitoreo de signos vitales denominado **Nexus Medical**. La rutina `setup_mpu_trap()` configura una región trampa en la dirección `0x20010000` (TRAP_ADDRESS), marcándola como no accesible mediante la MPU. Durante los primeros 15 segundos el sistema opera con normalidad, simulando el "latido" del dispositivo a través del parpadeo del LED integrado. Transcurrido ese tiempo, una tarea atacante intenta escribir en dicha dirección de forma deliberada, disparando un HardFault que el sistema debe interceptar y del que debe recuperarse según la estrategia implementada en `recovery_landing_zone()`.
Para demostrar el funcionamiento de cada reto se grabó un video en lugar de capturas estáticas, ya que los comportamientos clave son dinámicos: el cambio de frecuencia del LED, la desconexión momentánea del puerto `ttyACM0` y el flujo de mensajes en tiempo real son evidencia que una imagen fija no comunica con la misma claridad.

Prueba 0: Análisis de Rollback (Implementado)

El código base se compiló y se cargó en la Raspberry Pi Pico, el LED comienza a parpadear, después, al conectarnos a la terminal con Minicom se observa que transcurridos 15 segundos, el sistema imprime el mensaje de intento de escritura seguido del reporte de incidente, para luego retomar el parpadeo al mismo ritmo sin perder en ningún momento la conexión USB. En el video se aprecia este ciclo repitiéndose de forma continua, con la terminal siempre activa.

Líneas que registran el contexto del error antes de salvar al sistema

La primera es la llamada a `log_incident` dentro del HardFault Handler en C:

```c
/**
 * @brief C-part of the fault handler.
 */
void HardFault_Handler_C(uint32_t *stack_frame) {
    log_incident(stack_frame[6]); //PC esta en el indice 6 del stack frame
    stack_frame[6] = (uint32_t)recovery_landing_zone;
}
```

Código 1: Captura del *Program Counter* en el handler

Cuando ocurre una excepción en `ARM Cortex-M`, el hardware apila automáticamente ocho registros en este orden: `r0, r1, r2, r3, r12, lr, pc, xpsr`, por ello el índice `[6]` corresponde exactamente al *Program Counter*, es decir, a la dirección de la instrucción que causó la violación de memoria.

La segunda es la rutina `log_incident` en sí, que registra el estado antes de que el sistema tome cualquier acción de recuperación:

```c
/**
 * @brief Clinical Incident Logger.
```

```c
 */
void log_incident(uint32_t pc) {
    fault_count++;
    uint32_t time = to_ms_since_boot(get_absolute_time());
    printf("\n[!] ==========================================\n");
    printf("[!]   NEXUS MEDICAL: INCIDENT REPORT #%u          \n",
   fault_count);
    printf("[!]   TIME: %u ms | ATTACKER PC: 0x%08X      \n", time, pc);
    printf("[!]   STATUS: ISOLATING THREAT...                \n");
    printf("[!] ==========================================\n");
}
```

Código 2: Registro de telemetría antes del rollback

Esta función guarda el número de fallo acumulado, la marca de tiempo en milisegundos y la dirección exacta del *Program Counter* en el momento del ataque.

En un sistema de producción, estos datos se escribirían en memoria no volátil (Flash o EEPROM) antes del rollback, formando el "testamento" del sistema. Después, `longjmp(recovery_point, 1)` transfiere la ejecución de regreso al punto del `setjmp` en `main`, donde `start_time` se reasigna y el ciclo de 15 segundos comienza de nuevo.

Evidencia: ***Evidencia Reto 0 Práctica 7***

Reto 1: Degradación Elegante (Safe Mode)

Aquí cabe aclarar algo, el manual indica comentar el `longjmp` y activar la bandera `safe_mode_enabled` para que el procesador continúe ejecutando el código descendente en `recovery_landing_zone`. Sin embargo, durante la implementación se comprobó que eliminar el `longjmp` sin más provoca un comportamiento incorrecto: al retornar de la función, el procesador regresa al `while(1)` interno de `main` con la variable `start_time` aún vencida, lo que dispara un nuevo HardFault de inmediato antes de completar siquiera un parpadeo, generando un ciclo infinito de fallos con el LED apagado.

La solución que se implementó fue mantener el `longjmp` pero con la bandera activa **antes** del salto, de modo que al volver al `setjmp`, `start_time` se reasigna al tiempo actual y `blink_speed` ya vale 1000 ms en lugar de 200 ms:

```c
/**
 * @brief THE RESILIENCE HUB (Revised Lab Template).
 */
void recovery_landing_zone(void) {
    gpio_put(LED_PIN, 1); // Clinical Fault LED
    printf("[RECOVERY: DEGRADATION] Switching to safe clinical mode...\n")
   ;
    printf("[RECOVERY] Non-critical functions disabled. Core monitoring
   active.\n");

    safe_mode_enabled = true; //Lo debemos activar antes del salto
```

```

    for (volatile uint32_t i = 0; i < 8000000; i++) { __asm volatile ("nop"); }

    longjmp(recovery_point, 1); //Reinicia start_time y regresa al main
}
```

Código 3: Degradación Elegante: bandera activa antes del longjmp

En el video se observa que el LED parpadeaba rápido durante los primeros 15 segundos, se apagó un instante al producirse el ataque y al reaparecer lo hizo a un ritmo notablemente más lento, manteniéndose así en todos los ciclos siguientes. La variable `safe_mode_enabled` es `volatile` y global, por lo que su valor persiste entre iteraciones del `while(1)` externo. El sistema sobrevive al fallo y continúa operando, pero con sus funciones no esenciales desactivadas.

Evidencia 1: ***Evidencia Reto 1 (Error) Práctica 7***

Evidencia 2: ***Evidencia Reto 1 (Correcto) Práctica 7***

Reto 2: Micro-Reinicio de Tarea (Task Respawning)

En este reto el objetivo es que el sistema, tras interceptar el fallo, reinicie únicamente las variables internas de la simulación y retome la operación normal como si la tarea nunca hubiera fallado, sin activar el modo seguro ni alterar la velocidad del LED.

El problema central es que `start_time` es una variable local del `while(1)` interno de `main` y no es accesible desde `recovery_landing_zone`. La solución fue introducir una variable `sim_start_time` que el handler puede escribir, junto con una bandera `sim_reset_requested` que indica a `main` que debe usar ese nuevo valor al regresar del `longjmp`:

```
volatile uint32_t sim_start_time = 0;
volatile bool sim_reset_requested = false;

void recovery_landing_zone(void) {
    gpio_put(LED_PIN, 1);
    printf("[RECOVERY: RESPAWN] Fault detected. Re-initializing monitor task...\n");

    sim_start_time = to_ms_since_boot(get_absolute_time()); // Nuevo punto de inicio
    sim_reset_requested = true;

    for (volatile uint32_t i = 0; i < 8000000; i++) { __asm volatile ("nop"); }

    printf("[RECOVERY: RESPAWN] Task re-initialized. Resuming heartbeat...\n");
    longjmp(recovery_point, 1);
}

// En main, al regresar del longjmp:
```

```
uint32_t start_time = sim_reset_requested
    ? sim_start_time
    : to_ms_since_boot(get_absolute_time());
sim_reset_requested = false;
```

Código 4: Task Respawning: reinicio de variables de simulación

En el video se aprecia que, tras el ataque, el LED se apaga un instante y luego retoma el parpadeo rápido, idéntico al de arranque. La terminal muestra el reporte de incidente seguido del mensaje de reinicio de tarea y, aproximadamente 15 segundos después, el ciclo se repite. Esto confirma que la "tarea" renació limpia con un contador de tiempo fresco, sin que el resto del sistema se viera afectado.
Evidencia: ***Evidencia Reto 2 Práctica 7***

Reto 3: Fail-Safe (Hardware Reset por Watchdog)

Este reto representa el escenario donde el fallo es irrecuperable y la única acción válida es reiniciar el hardware por completo, en la implementación hecha utilizamos la API del SDK de la Pico para invocar el *Watchdog Timer* de forma intencional:

```
void recovery_landing_zone(void) {
    gpio_put(LED_PIN, 1);
    printf("[RECOVERY: FAIL-SAFE] Critical failure detected.\n");
    printf("[RECOVERY: FAIL-SAFE] State is unrecoverable. Performing hard reboot...\n");
    printf("[RECOVERY: FAIL-SAFE] USB will disconnect. System restarting ...\n");
    stdio_flush();

    for (volatile uint32_t i = 0; i < 2000000; i++) { __asm volatile ("nop"); }

    watchdog_reboot(0, 0, 0); //Reset fisico inmediato

    while (1) { __asm volatile ("nop"); } //Nunca se ejecuta
}
```

Código 5: Fail-Safe: reinicio físico mediante watchdog

Se añadió además una verificación al inicio de `main` para detectar si el arranque provino de un reset por *watchdog*:

```
if (watchdog_caused_reboot()) {
    printf("[SYSTEM] *** REBOOT DETECTED: System recovered from watchdog reset ***\n");
}
```

Código 6: Detección de reboot por watchdog al arrancar

En el video se observa que al producirse el ataque la terminal imprime los tres mensajes de *Fail-Safe*, el puerto `ttyACM0` se desconecta momentáneamente (evidencia del reset eléctrico

real del RP2040) y Minicom lo reconecta de forma casi instantánea al completarse la secuencia de arranque del chip. La variable `fault_count` vuelve a `#1` en cada ciclo, a diferencia de los retos anteriores donde se acumulaba, confirmando que toda la `SRAM` fue borrada por el reset físico.
Evidencia: ***Evidencia Reto 3 Práctica 7***

Reflexión:
¿por qué un `while(1)` en el HardFault Handler es peligroso?

Una práctica común en código de ejemplo o en entornos educativos es colocar un `while(1)` dentro del *HardFault Handler* como mecanismo de "detención segura", aunque parece inofensivo, en dispositivos reales esta decisión tiene consecuencias graves.

En primer lugar, el sistema queda **completamente bloqueado**, ninguna interrupción de mayor prioridad puede sacarlo de ese bucle porque el *HardFault Handler* corre al nivel de prioridad más alto del sistema. El *Watchdog Timer*, si existe, eventualmente expirará y forzará un reinicio, pero ese reinicio ocurre sin ningún registro del fallo, sin apagar actuadores de forma controlada y sin ejecutar ninguna rutina de limpieza.

En segundo lugar, los **actuadores físicos quedan en su último estado**, por ejemplo en robótica, aviónica o en equipos médicos, esto significa que un motor puede seguir girando, una válvula puede permanecer abierta o un estimulador eléctrico puede continuar activo mientras el procesador está atascado en el bucle. La consecuencia puede ser un daño físico al sistema o a las personas que lo operan.

En tercer lugar, se **pierde toda oportunidad de diagnóstico**, el handler con `while(1)` no registra el *Program Counter*, no vuelca el stack, no escribe nada en memoria no volátil. Los desarrolladores quedan en la misma situación descrita en el **Caso 5 de la Fase A**, donde el sistema falló, pero no dejó ningún rastro forense.

La alternativa correcta, demostrada a lo largo de esta práctica, es que el *HardFault Handler* ejecute siempre al menos una rutina de Caja Negra (registro del PC y del stack en memoria no volátil), seguida de la estrategia de recuperación apropiada para el contexto, ya sea un rollback por software, una degradación elegante, un respawn de tarea o un reinicio controlado por *watchdog* que garantice que los actuadores queden en un estado seguro conocido antes de que el chip se reinicie.

Evidencias

Evidencia Práctica 7 Completo

Evidencia Práctica 7 (Reto 0)

Evidencia Práctica 7 (Reto 1)

Evidencia Práctica 7 (Reto 2)

Evidencia Práctica 7 (Reto 3)

Conclusiones

A lo largo de ambas fases pudimos apreciar que no existe una estrategia de recuperación universalmente correcta. La elección adecuada depende de la criticidad del subsistema afectado, la naturaleza del fallo y el costo de perder el estado acumulado.

En la **Fase A** vimos cómo cada escenario exige una respuesta distinta: la *Degradación Elegante* es viable cuando el fallo ocurre en un subsistema prescindible y la MPU garantiza que estuvo contenido; el *Reinicio Duro* es la única opción responsable cuando el núcleo de control físico está comprometido y continuar operando representa un peligro real; el *Checkpointing y Rollback* cobra sentido cuando el estado acumulado tiene un costo computacional tan alto que descartarlo es inaceptable. Finalmente, el caso del patín eléctrico nos recordó que el *Patrón de Caja Negra* no es una estrategia de recuperación sino una obligación previa a cualquiera de ellas, pues sin un registro forense del fallo, el sistema sobrevive pero el equipo de desarrollo queda ciego.

En la **Fase B** comprobamos con hardware real que esas abstracciones producen comportamientos medibles y distintos entre sí.
La **Prueba 0** nos mostró que recuperarse de un HardFault no implica reiniciar el hardware: mediante `longjmp` el sistema regresó a un estado estable sin interrumpir la conexión USB. El **Reto 1** eliminar el `longjmp` ingenuamente genera un ciclo infinito de fallos porque `start_time` permanece vencido; la solución implementada fue activar la bandera de modo seguro *antes* del salto, no después. El **Reto 2** añadió la necesidad de variables `volatile` globales para que el handler pudiera reiniciar el estado local de la tarea, evidenciando que la recuperación localizada exige que el estado a limpiar sea accesible desde el contexto de la excepción. El **Reto 3** fue el más contundente: la desconexión momentánea del puerto `ttyACM0` y el reinicio de `fault_count` a `#1` confirmaron que el reset por Watchdog borra la SRAM por completo, algo que ninguna estrategia por software puede garantizar.

En conjunto, la MPU no es un mecanismo de seguridad aislado sino la base que hace posibles todas estas estrategias, sin el aislamiento que provee, ninguna decisión tomada dentro del *HardFault Handler* puede considerarse verdaderamente segura.